· 2026 年数通第八届爱迪生创新大赛 · 入围作品 ·

基于 AI Agent 的个人本地多模态文档智能问答系统

doc2md × DocBase × AGENTS.md —— 让沉睡的本地文档“开口说话”

摘 要

个人本地沉淀了大量 PDF、Word、PPT 等多模态文档，一旦忘记存放位置与文件名，仅凭模糊的内容记忆极难找回。本创意通过 AI Agent（CodeAgent）自动开发 Python 脚本，结合 RAG、Embedding、Reranker、Chat、VL 等大模型能力，预解析本地文档并构建 RAG 数据库，再以 AGENTS.md 指导 Agent 消费该库，实现“内容即入口”的智能问答与精准溯源。

技术栈 CodeAgent RAG Embedding Reranker Chat LLM VL 模型

公司装备部提供模型支持：Embedding（**Qwen3-Embedding-8B**）· Reranker（**Qwen3-Reranker-8B**）· Chat（**GLM-5.1**）· VL（**Qwen3.6-27B-VL**）

✓ 全本地运行 ✓ 图片可检索 ✓ 回答可溯源 ✓ 模型可替换

创意提交人：**井永乐 | 00587807** · 转发开发部 [ICT BG]

2026-07-30

ARCHITECTURE

系统总体架构

三层解耦架构：文档解析层 → 知识库层 → Agent 消费层

全本地运行 · 仅调用公司内部模型服务 · 数据不出本机

① PARSING 文档解析层

doc2md · 多模态文档 → 结构化 Markdown

codehub-y.huawei.com/j00587807/doc2

- **格式转换** — PDF/Word/PPT→MD
- **OCR 提取** — 位图 + 矢量图
- **VL 多模态解析** — Qwen3.6-27B-VL 释义
- **Markdown 生成** — 图片描述写入正文
- **溯源元信息前置** — 文档名 · Page · Figure Title
- **原始链接关系** — 可逆向定位原文

产出：**结构化 Markdown + 图片 AI 释义**

② RETRIEVAL 知识库层

DocBase · RAG 数据库

codehub-y.huawei.com/j00587807/DocB...

- **Chunk 切片** — 语义分段 · 多文档
- **向量化** — Qwen3-Embedding-8B
- **混合检索三路召回** — 稠密 + BM25 + BGE-M3
- **RRF 融合** — 多路结果归并
- **Reranker 重排** — Qwen3-Reranker-8B
- **TOP-K 文本块** — 含溯源元数据

能力：**build（进度条）/ query / test 三模式**

③ AGENT Agent 消费层

AGENTS.md 驱动 CodeAgent 问答

CLI / Web / Service

- **AGENTS.md 知识入口** — 指导 Agent 如何消费 RAG
- **意图理解 · RAG 查询** — 自动构建 query 检索知识库
- **GLM-5.1 综合推理** — 多文档交叉 · 生成回答
- **结构化产出** — Mermaid 图 / 表格 / 摘要
- **溯源输出** — 定位原始文档与页码

扩展：**后台 Service · Web 应用（可选）**

解析入库 → 混合检索 → Agent 问答：三层各司其职，接口清晰

依赖方向：上层 → 下层 · 仅经 models 契约通信

PARSING · 产物契约

document.md + assets/ + manifest.json

每张图带 Page / Fig / 标题，可回链原文

RETRIEVAL · 检索契约

TOP-K + 三路分数 + index version

build / query / test 三模式，可增量、可回滚

AGENT · 回答契约

answer + citations + rounds_used

无据不答，每条结论带原文定位

LOGICAL VIEW

DocQA 文档问答系统 — 逻辑视图

AGENTS.md 驱动 Agent 消费 · 三路混合检索 · 图文一体解析 · 全链路溯源

核心问答链路 · 模型服务调用 · 可选扩展

用户入口层 CLI 命令行问答 `CodeAgent + AGENTS.md`

Web 网页问答 B/S · 无头 Agent — 可选

模型应用 仅调用内部模型服务 · 数据不出本机

Agent 消费层 · DOCQA.AGENT-DRV

AGENTS.md 驱动

意图分析 → RAG 查询 → 充足性自评 → 综合推理 → 结构化回答

意图分析	RAG 查询	充足性自评	综合推理	结构化回答
拆解检索词 判断分类	调用 DocBase 检索	不足时多轮 补充检索	GLM-5.1 跨文档交叉	Mermaid / 表格 / 溯源

约定： 必须先检索 · 无据不答 · 回答强制附原文链接 / 文档名 / 页码 / 图号

RAG 检索引擎 · DOCQA.DocBase

build 建库 · query 检索 · test 评估

- ① **稠密向量检索** Qwen3-Embedding 语义匹配
- ② **BM25 关键词检索** 精确术语 · 型号 / 命令 / 接口名
- ③ **BGE-M3 稀疏检索** 学习型稀疏向量 · 兼顾词面权重

三路并行召回 · 互补检索盲区 · **任一路异常自动降级可用**

→ RRF 融合（倒数排序 / 归并多路）→ 交叉编码精排（Reranker-8B / TOP-K 筛选）→ TOP-K 文本块（携带溯源 / 元数据）→ 供 Agent（综合回答）

test 模式：召回率 / 命中率 / MRR 量化评估，支撑参数调优

知识资产层（本地持久化 · 增量更新）

Markdown 语料库	溯源图片库
标题层级 · 图文一体	文档名 · Page 图序 · 原图
RAG 三路索引	**文档注册表**
稠密向量 + BM25 + 稀疏	分类管理 · 增量更新

索引格式版本化 · 支持增量更新与回滚

index.json · vectors.json · sparse.json · manifest.json

扫描目录 → 哈希比对 → 增量重建 → 版本回滚

5	32	3	0.6MB
文档	文本块	图片	索引

文档解析引擎 · DOCQA.Doc2md

多格式输入（PDF/Word/PPT）→ 格式统一（转 Markdown）→ OCR 提取（位置 / 矢量图）→ VL 释义（图文一体写入）

溯源命名： 文档名_Page图序_Figure标题，可逆向定位原文；矢量图先渲染为位图再识别，释义写入原图引用处。

> 图 2-1（VL 释义）：分机注册 → 鉴权 → 号码分析 → 中继选路 → 媒体协商 → 通话建立…

PDF · DOCX · PPTX · MD · TXT

页码：PDF / PPT = exact；DOCX = estimated（明示）

内部模型服务

装备部 · 端点可配置

VL	Qwen3.6-27B-VL	图片深度释义（模块 / 箭头 / 编号语义）
EM	Qwen3-Embedding-8B	文本向量化（稠密检索）
RR	Qwen3-Reranker-8B	交叉编码精排（TOP-K 筛选）
CH	GLM-5.1	多文档交叉推理 · 回答生成

端点级配置注入，可替换 / 可 Mock；版本升级不影响产物格式。

可选扩展

不启用不影响核心功能

- **后台 Service** — 定时 / 增量更新 · 监控目录管理
- **Web 应用** — B/S 网页问答 · Mermaid 图形渲染

可观测

可复盘

trace	每轮 query / 命中数 / 覆盖率 / 耗时
warnings	降级原因、缺失图片、端点异常
watch.log	增量更新事件，JSONL 可审计

部署边界

本地优先

本地文件系统	Web 127.0.0.1
内部模型端点	Mock 可离线

- 文档 / 索引 / 图片不出本机
- API Key 只读环境变量，不写日志

质量快照

演示集

1.00	19	2ms
recall@6	测试通过	Mock 检索

全链路溯源 · 无据不答

回答 → 文本块 → Markdown → 原始文档

原文链接 · 文档名 · 页码 · 图号，一键跳回原文。

一次解析 · 三路检索 · Agent 消费 —— 知识问答流水线全本地闭环

DEVELOPMENT VIEW

DocQA 文档问答系统 — 开发视图

Python 工程组织 · 变更点 ■ 模块映射 · 分层依赖单向 · 契约稳定

五个变更点五个目录 · 换 Agent / 换模型不改约定与契约

docqa 工程目录结构

Python · 数据与约定分离

```
docqa/
├── doc2md/              # 变更点 1 · 解析
│   ├── parser.py            # 统一入口 + pdf/word/ppt
│   ├── image_extractor.py   # 图片保存 / 尺寸
│   ├── vl_client.py         # VL 释义适配
│   ├── naming.py            # 溯源命名
│   └── models.py            # ImageInfo / ParseTask
├── docbase/             # 变更点 2 · 检索
│   ├── chunker.py           # 语义切片
│   ├── indexer.py           # build / 增量 / 三路索引
│   ├── retriever.py         # query / RRF / rerank
│   ├── evaluator.py         # test: recall@k / MRR
│   └── models.py            # Chunk / RagIndex / QueryResult
├── agent/               # 变更点 3 · 消费
│   ├── answer.py            # 问答主流程
│   └── models.py            # Answer / Citation
├── service/             # 变更点 4 · 可选
│   ├── watcher.py           # 目录监控 / 增量更新
│   └── models.py            # WatchConfig
├── web/                 # 变更点 5 · 可选
│   ├── server.py            # stdlib HTTP 服务
│   ├── models.py            # WebQueryResponse
│   └── static/index.html    # B/S 前端
├── llm.py                   # 四类端点 + Mock
├── cli.py                   # parse/build/query/ask/eval
└── utils.py
tests/                   # test_doc2md / docbase / agent / web
AGENTS.md                # 知识消费约定
```

运行时： Python 3.9+ · 标准库 · PDF 可选 pypdf · Web 无重型框架

doc2md = 变更点 1 | docbase = 变更点 2 | agent = 变更点 3 | service = 变更点 4 | web = 变更点 5

分层依赖

自上而下单向

入口层：web / service
无头 Agent 调用 · 定时增量触发

↓

Agent 层：agent
AGENTS.md 约定 · 意图分析 · 多轮检索编排

↓

检索层：docbase
切片 / 建库 / 三路召回 / 精排 / 评估

↓

解析层：doc2md
格式转换 / OCR / VL 释义 / 溯源命名

↓

模型端点层（外部服务 · 可 Mock）
VL · Embedding · Reranker · GLM-5.1

可测性设计

- **docbase 内置 test 模式：** recall@k / hit_rate / MRR
- **四类模型服务：** 统一端点注入，UT 用 Mock 替换
- **溯源链路确定性：** 回答 → Chunk → MD → 原文，端到端断言
- **doc2md 产物图文互证：** Mermaid 重绘 vs 原图对照
- **当前：** 19 个单元/集成用例全部通过

doc2md	Markdown / PDF / DOCX / PPTX / TXT · 图片命名与回链
docbase	三路召回 · 精确术语 · 分类过滤 · 增量 · 评估
agent	流程 / 命令 / 无据拒答 / 契约字段 / 意图
web	静态页 · stats · ask · 原文接口

核心接口（Spec 契约）

```
doc2md
parse_document(src, out, vl_opts)
batch_parse(dir, recursive, types)

docbase
build_index(md_dir, incremental)
query(index, q, top_k, category)
eval_retrieval(index, test_set)

agent
answer_question(q, index, max_rounds)

service / web（可选）
register_watcher(cfg) / update_single_doc(doc)
handle_query(req) → WebQueryResponse
```

兼容：字段只增不改 · 索引可回滚 · CLI 向后兼容

AGENTS.md 四类约定

数据与代码分离

① 存在性声明
本仓库配备 DocBase 库，先检索再回答

② 查询策略
分类缩小范围；名词走 BM25，原理走向量

③ 溯源强制
附链接/文档名/页码/图号；无据不答

④ 产出形式
流程类重绘 Mermaid，与原图互证

兼容与回滚
字段只增不改 · 索引版本可回滚 · CLI 参数向后兼容

运行： ./run.sh demo · make test · ./run.sh serve

关键数据契约（models.py） doc2md：ImageInfo / ParseTask / ParseResult | docbase：Chunk / RagIndex / QueryResult | agent：answer / format / citations / rounds_used | WebQueryResponse 与 answer_question 对齐

ROADMAP

实施路径：五步渐进，逐步演进

全部通过 CodeAgent Vibe Coding 开发 —— AI 写代码，人做决策

1–3 步：核心闭环 · 4–5 步：团队与领域演进

核心闭环 doc2md → DocBase → AGENTS.md 已完成 Demo | 可选扩展 Service → Web 已提供可运行实现

1 文档格式转换工具 doc2md

基础必做 · 已完成 Demo

多模态文档 → 结构化 Markdown

- PDF/Word/PPT 一键转 Markdown
- OCR 提取文档内图片（位置 + 矢量图）
- 调用 VL 模型解析图片内容并写入 MD
- 保留对原始文档的链接关系
- 图片命名含文档名/Page/Figure Title，支持逆向溯源

Demo：codehub-y.huawei.com/j00587807/doc2md

验收： 5 类格式 · 图片命名可逆 · manifest 完整

→

2 RAG 检索工具 DocBase

基础必做 · 已完成 Demo

三路混合召回 · RRF · Reranker 精排

- Markdown 切片（Chunk）+ 向量化入库
- 三路混合召回：稠密向量 + BM25 + BGE-M3 稀疏
- RRF 融合 + Reranker 重排，筛选 TOP-K
- build（进度条）/ query / test 三命令
- 支持多文档、文档分类管理

Demo：codehub-y.huawei.com/j00587807/DocBase

验收： 三路召回 · recall@6 · 增量建库

→

3 AGENTS.md 知识消费约定

基础必做 · 已完成 Demo

本创意的核心差异点 · 约定驱动消费

- 指导 CodeAgent 消费 RAG 向量库与 Markdown 原文
- 约定查询策略：何时检索、如何组合、如何引用
- 给出符合预期的回答 + 溯源，快速定位原始文档
- 示例：DocBase/CLAUDE.md（参考配置文件）
- 换 Agent / 换模型，约定与契约不变

示例：DocBase/CLAUDE.md（参考配置文件）

验收： 四类约定 · 无据不答 · 引用可回原文

4 后台 Service 自动化运维

可选扩展 · 团队级能力

知识库持续保鲜，无需人工干预

- 定时/增量更新指定目录、指定文档类型
- 文档管理、向量化数据库管理、图片解析管理
- 支持新增监控目录、单文档解析、递归搜索

文档库持续保鲜，无需人工干预；知识库随文档沉淀自动演进

验收： 变更检测 · 增量重建 · 事件日志

→

5 B/S 架构 Web 应用

可选扩展 · 领域级能力

从个人工具到团队级、领域级应用

- 脱离 CodeAgent 命令行，网页图形界面
- 后台调用 CodeAgent 无头模式：claude -p / nga run
- 确保回答质量，图形渲染更直观

降低使用门槛、扩大覆盖人群；从个人工具升级为团队级、领域级应用

验收： 网页问答 · 引用卡片 · 原文抽屉

5	3	100%	19
类输入格式 · 图文可解析	路混合召回 · 互补盲区	回答带引用 · 可回原文	测试用例 · 全部通过

doc2md： 多模态文档解析引擎

第一步产出 · 让图片里的知识变得可检索

PDF / Word / PPT → 结构化 Markdown + 图片 AI 释义

解析流水线

六步 · 图文一体

① **多格式输入**
PDF · Word · PPT 等多模态文档

② **格式统一转换**
全量转为 Markdown 标准结构；后续可平行扩展 Visio、Drawio 等

③ **图片提取**
OCR 技术提取位图 + 矢量图

④ **VL 模型释义**
Qwen3.6-27B-VL 解析图片内容

⑤ **图片描述写入 MD**
AI 释义随图入库，图文一体

⑥ **结构化 Markdown 输出**
供 RAG 切片与 Agent 消费

溯源命名规范（关键设计）

`文档名_Page图序_Figure标题`

保留对原始文档的链接关系；任意图片可逆向定位到原文档、页码与图题。

写回示例

```
![图 2-1](assets/..._Page2_Fig1_分组转发呼叫接续流程.png)
> 图 2-1（VL 释义）：注册 → 鉴权 → 号码分析 → 中继选路 → 媒体协商 → 通话建立
```

输入格式	PDF · DOCX · PPTX · MD · TXT
页码策略	PDF / PPT = exact；DOCX = estimated
失败降级	缺图保留引用；EMF/WMF 告警；VL 失败回退 Mock
产物清单	document.md · assets/ · manifest.json
质量校验	图文互证 · 命名可逆 · 页码可回

PDF	DOCX	PPTX
pypdf · page exact · 嵌入图	OOXML · page estimated · 表格/图片	幻灯片 · page exact · 一页一图

data/parsed/<doc>/ · document.md · assets/ · manifest.json

doc2md 把“看不懂的图”变成“读得懂的文”，**让多模态知识第一次进入可检索、可问答的领域。**

能力矩阵与设计考量

全格式 · 双形态 · 深释义

全格式覆盖
PDF / Word / PPT 统一入口，后续可平行扩展 Visio、Drawio 等。

双形态图片
位图直连 OCR；矢量图先渲染为位图再识别，流程图/架构图全覆盖。

VL 深度释义
不止 OCR 文字，还理解图中模块、箭头、编号等结构化语义。

图文一体入库
图片释义写在原图引用处，切片后图文语义不割裂，检索上下文完整。

命名可逆 · 释义可读 · 页码可回 · 切片不割裂

parser.py 统一入口 + 类型分发	**image_extractor.py** 保存 / 尺寸 / 命名
vl_client.py 端点注入 + Mock	**manifest.json** 图片清单 / 统计 / 告警

为什么这一步是整个系统的基石?

- 技术文档中约 30%–50% 的关键信息以图片承载（架构图/流程图/时序图/状态机）。
- VL 释义与原文档名、页码、图题绑定，回答时可“图文互证”并回溯原文。
- 传统 RAG 只切文本，图片信息全部丢失——本工具将图片转化为自然语言描述。
- Markdown 是 Agent 最友好的知识格式，为后续 RAG 切片提供最佳载体。

30%–50% 的图片信息，不再沉默。

扫描件	需 OCR 兜底，先文字后 VL
矢量图	当前跳过并告警，可接 PyMuPDF 渲染
原则	描述不做结论，结论仍由文本证据支撑

DocBase：三路混合检索 RAG 引擎

第二步产出 · 稠密向量 + BM25 + BGE-M3 稀疏检索，RRF 融合，Reranker 精排

build 入库 · query 检索 · test 质量自评

build 模式 · 知识入库流水线

进度条可视化

Markdown 文档集
多文档 · 支持分类

→ 语义切片 Chunk
按结构切分保留上下文

→ Embedding 向量化
Qwen3-Embedding-8B

→ 稀疏索引构建
BM25 词频 + BGE-M3 稀疏

→ 向量数据库落盘
本地持久化 · 增量更新

build ——— 解析 → 切片 → 向量化 → 索引 → 落盘 · 进度条实时显示

三种工作模式：build 构建知识库（进度条显示）/ query 查询 / test 检索效果自测 · 支持多文档、文档分类管理

query 模式 · 三路召回 → 融合 → 精排 → TOP-K

任一路异常自动降级可用

① 稠密向量检索
语义相似度匹配（Embedding）

② BM25 关键词检索
精确术语命中（型号/命令/接口名）

③ BGE-M3 稀疏检索
学习型稀疏向量，兼顾词面权重

→ RRF 融合
倒数排序
融合多路结果

→ Reranker 精排
Qwen3-Reranker-8B
交叉编码

→ TOP-K 文本块
携带溯源
元数据输出

→ 供 Agent 综合回答
test 模式
量化评估检索质量

为什么要三路混合?

纯向量检索对“SD5993”“FE_HEART_Start”这类精确术语容易模糊命中；纯关键词检索无法理解“分机转发是怎么走的”这类语义问题。**三路召回互补盲区 + RRF 消除量纲差异 + Reranker 交叉编码精排**——兼顾语义理解与术语精确，这是检索质量的关键保障。

一次真实检索（演示索引）

query：SD5993 单板怎么复位？命令是什么？ → 稠密 0.219 · BM25 7.63 · 稀疏 0.07 → RRF 融合 → Reranker 1.00 → TOP-1《03_SD5993单板命令手册》Page 2 · Fig 1。

build
chunk=1200 · overlap=120 · 增量复用

query
三路各 top-50 · RRF k=60 · 精排 top-24

test
recall@6 · hit_rate · MRR · JSON 报告

索引三件套
index.json · vectors.json · sparse.json
版本化落盘，支持增量与回滚

评估指标
recall@k · hit_rate · MRR
keyword_hit_rate · 平均耗时

降级策略
单路失败 → 其余两路继续
Reranker/Chat 失败 → RRF / 抽取式

```
build
./run.sh build data/parsed --incremental
```

```
query
./run.sh query "分机怎么转发？" --top-k 6
```

```
test
./run.sh eval --index data/index
```

build 建库　query 检索　test 评估

AGENTS.md：让 Agent 学会“查资料”

第三步产出 · 本创意的核心差异点 —— 用约定文件驱动 Agent 消费 RAG

RAG 不会自动被使用，必须有人“告诉”Agent 它的存在、用法与边界

AGENTS.md 是什么

- CodeAgent（Claude Code / NGA 等）启动时**自动加载的项目级指令文件**
- 声明本仓库存在 **DocBase RAG 知识库**，并约定何时查、怎么查、如何引用
- 把“人的检索经验”固化为 **Agent 可执行的工作流约定**
- 用户无需学习任何命令 —— 像跟同事对话一样提问即可

关键洞察

RAG 数据库本身不会自动被使用 —— 必须有人“告诉”Agent 它的存在、用法与边界。**AGENTS.md 就是这座桥梁**，也是本创意区别于传统 RAG 产品的核心创新点。

生效机制

约定先于回答

启动时	CodeAgent 自动加载 AGENTS.md，知道库存在
检索前	按问题类型选择 BM25 / 向量 / 分类过滤
回答前	校验 [n] 与原文定位，不足则无据不答

传统 RAG	检索接口是“可选能力”，Agent 不知道库存在
DocQA	约定是“默认工作方式”，每次回答先检索

落地效果

回答有据
每条结论 [n] + 文档名 / 页码 / 图号

无据会拒
域外问题明确说不知道，不编造

换模型不换约定
GLM / Mock 输出同一契约

Agent 问答工作流

S 形：提问 → 检索 → 综合 → 呈现 → 溯源

接收提问 自然语言问题	**意图分析** 拆解为检索词	**执行 RAG 查询** 调用 DocBase	**补充检索** 信息不足时多轮
筛选整合 TOP-K 文本块	**GLM-5.1 综合** 多文档交叉推理	**结构化呈现** Mermaid / 表格	**溯源输出** 定位原始文档

S 形工作流： 提问 → 检索 → 综合 → 呈现 → 溯源，全程由 AGENTS.md 约定驱动；Agent 可自主判断“答案够不够好”，不足时自动发起第二轮检索。

典型约定（节选示意）

知识库存在性声明 “本仓库配备 DocBase RAG 库，回答领域问题前必须先检索”	**查询策略约定** “按文档分类缩小范围；技术名词走 BM25，原理问题走向量检索”
溯源强制要求 “回答必须附原文链接与定位信息（文档名/页码/图号），无据不答”	**产出形式约定** “流程类问题用 Mermaid 绘图辅助说明，并与原始图片对照验证”

把检索经验写进约定，让 Agent 的每次回答都有据可查。

answer　format　citations　rounds_used

answer_question 契约：字段只增不改，WebQueryResponse 对齐。

```
## 1. 存在性声明
- 本仓库配备 DocBase RAG 知识库，索引位于 data/index/
- 回答 FE_HEART / SD5993 / 分组转发 / 运维问题前，必须先检索
- 不得凭模型记忆直接回答，不得编造文档名、页码、图号
```

```
## 3. 溯源强制
- 每个结论后标注 [n]，附文档名 / 页码 / 图号
- 一个 [n] 只对应一个文本块，不同文档不得合并编号
- 无据不答：检索不足时直接说明，不用常识补全
```

不是提供一个检索接口，而是让 Agent 养成“先查资料”的默认工作方式。

DELIVERY

可运行交付物 · 一条命令跑通全链路

parse → build → ask → eval · 19 项测试 · CLI / Web / watcher 三入口

Mock 模式 · 离线可复现 · 真实端点可切换

DocQA · demo

```
$ ./run.sh demo
DocQA Demo · 多模态文档 → 知识库 → Agent 问答

① 解析示例文档（doc2md · Mock VL 释义）
   5 个文档 · 3 张图片

② 构建三路混合索引（DocBase）
   32 个文本块 · 版本 2026.10.24-xxxxxx

③ Agent 问答（AGENTS.md 约定驱动）
Q: 分机注册后，呼叫是怎么转发到中继的?
**结论**：分机注册后，呼叫接续按以下六个步骤执行：[1]
1. 分机注册 → 2. 鉴权 → 3. 号码分析
4. 中继选路 → 5. 媒体协商 → 6. 通话建立
**流程重绘（Mermaid）**
  A[分机注册] → B[鉴权] → C[号码分析] → D[中继选路] …
> 可与《02_分组转发设计》Page 2 · Fig 1 原图对照

④ 检索质量评估（test 模式）
   recall@6=1.0  hit_rate=1.0  MRR=1.0

$ ./run.sh serve --open        # B/S 问答
$ ./run.sh watch --src docs --once  # 增量
```





1.00	19	2 ms
recall@6 · 演示集	测试用例通过	Mock 检索耗时

- **代码** src/docqa · 标准库运行时
- **约定** AGENTS.md / CLAUDE.md
- **文档** 项目书 / 技术设计 / 实施验收 / 讲稿
- **汇报** Deck HTML + PDF · 10 页

指标为 5 篇演示文档 / 8 条问题 / Mock 链路结果，仅证明链路与口径；生产验收需接入真实端点并重建测试集。

THANK YOU

让每一份沉睡的本地文档
都能开口说话。

doc2md

把“看不懂的图”变成“读得懂的文”，让图片进入检索。

DocBase

三路召回 + RRF + 精排，兼顾术语精确与语义理解。

AGENTS.md

把检索经验写进约定，让 Agent 先查资料、再回答、必溯源。

谢谢各位评委 **Q & A**

井永乐 | 00587807 · 转发开发部 [ICT BG] · 2026-07-30